



ARTIFICIAL NEURAL NETWORKS (ANN) LAB 06

Lab Demonstrator: Shakeela Shaheen



LAB 06: IMPLEMENTING CONVOLUTIONAL NEURAL NETWORKS (CNN) FOR IMAGE CLASSIFICATION

LEARNING OBJECTIVES:

By the end of this lab, students will:

- Build and train a CNN model using TensorFlow/Keras.
- Implement convolutional and pooling layers.
- Apply activation functions in CNN.
- Perform image classification using CNN.
- Use Dropout and Batch Normalization for regularization.
- Compare optimizers (SGD vs Adam).
- Evaluate model using accuracy and loss curves.
- Detect overfitting using validation curves.
- Analyze training stability and convergence.

Convolution Neural Network

Convolutional Neural Networks (CNNs), also known as ConvNets, are neural network architectures inspired by how the human brain processes visual information and are widely used in computer vision tasks.

CNNs are a special type of deep learning model designed mainly for **image processing, image classification, object detection**, and **computer vision** tasks.

Why CNN Instead of ANN?

ANN Problem with Images

- ANN takes input as a 1-D vector.
- If an image is 100×100 with 3 color channels → **30,000 inputs**.
- ANN treats all pixels equally — **no spatial information**.

- Too many weights → slow and high chance of overfitting.

CNN Advantage

- CNN keeps the **spatial (2D) structure** of the image.
- Automatically detects:
 - Edges
 - Corners
 - Shapes
 - Patterns
 - Objects

CNN learns local features using **filters**, which makes it efficient and accurate.

Input 5×5

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

Kernel 3×3

0	1	2
2	2	0
0	1	2

1- Kernel

- Kernel is nothing but a filter that is used to extract the features from the images.
- The kernel is a matrix that moves over the input data, performs the dot product with the sub-region of input data, and gets the output as the matrix of dot products.
- Kernel moves on the input data by the stride value.

Feature Map 3x3

12	12	17
10	17	19
9	6	14

Convolution Output Size Formula

$$[\text{Output Size} = (\text{Input Size} - \text{Kernel Size}) + 1]$$

Or

$$[O = (I - K) + 1]$$

Where:

- (I) = Input dimension
- (K) = Kernel (filter) dimension
- (O) = Output feature map dimension

2- Stride

- The filter is moved across the image left to right, top to bottom, with a one-pixel column change on the horizontal movements, then a one-pixel row change on the vertical movements.
- The amount of movement between applications of the filter to the input image is referred to as the **stride**, and it is almost always symmetrical in height and width dimensions.
- The default stride or strides in two dimensions is **(1,1)** for the height and the width movement.

Input 5×5

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

Kernel 3×3

0	1	2
2	2	0
0	1	2

Feature Map 2x2

12	17
9	14

Output Size Formula with Stride (2)

For an input of size (I), kernel of size (K), and stride (S):

$$[\text{Output Size} = \left\lfloor \frac{I-K}{S} \right\rfloor + 1]$$

3- Padding / Border Problem Solver

- Padding is the best approach, where the number of pixels needed for the convolutional kernel to process the edge pixels are added onto the outside copying the pixels from the edge of the image.
- Fix the Border Effect Problem With Padding.

$$[\text{Output Size} = \left\lfloor \frac{I-K+2P}{S} \right\rfloor + 1]$$

$$\text{Output Size} = \left\lfloor \frac{5-3+2(1)}{1} \right\rfloor + 1$$

$$\text{Output Size} = 5 * 5$$

Output of Padding 5×5

6	14	17	11	3
14	12	12	17	11
8	10	17	19	13
11	9	6	14	12
6	4	4	6	4

4- Pooling Layer

- Used to reduce the size of feature maps.
- Pooling layers provide an approach to down sampling feature maps by summarizing the presence of features in patches of the feature map.

Input 5×5

6	14	17	11	3
14	12	12	17	11
8	10	17	19	13
11	9	6	14	12
6	4	4	6	4

Types:

1. Max Pooling (most common)

- Takes the largest value in a region.
- Good for keeping dominant features.

Feature Map 2x2

14	17
11	19

2. Average Pooling

- Takes average value in a region.

Feature Map 2x2

11.5	14.25
9.5	14.0

5- Flattening

- Once the pooled featured map is obtained, the next step is to flatten it.
- It involves transforming the entire pooled feature map matrix into a single column which is then fed to the neural network for processing.

6. Fully Connected Layer (FC Layer)

- Works same as normal ANN.
- Final classification happens here.
- Applies Softmax for multi-class outputs.

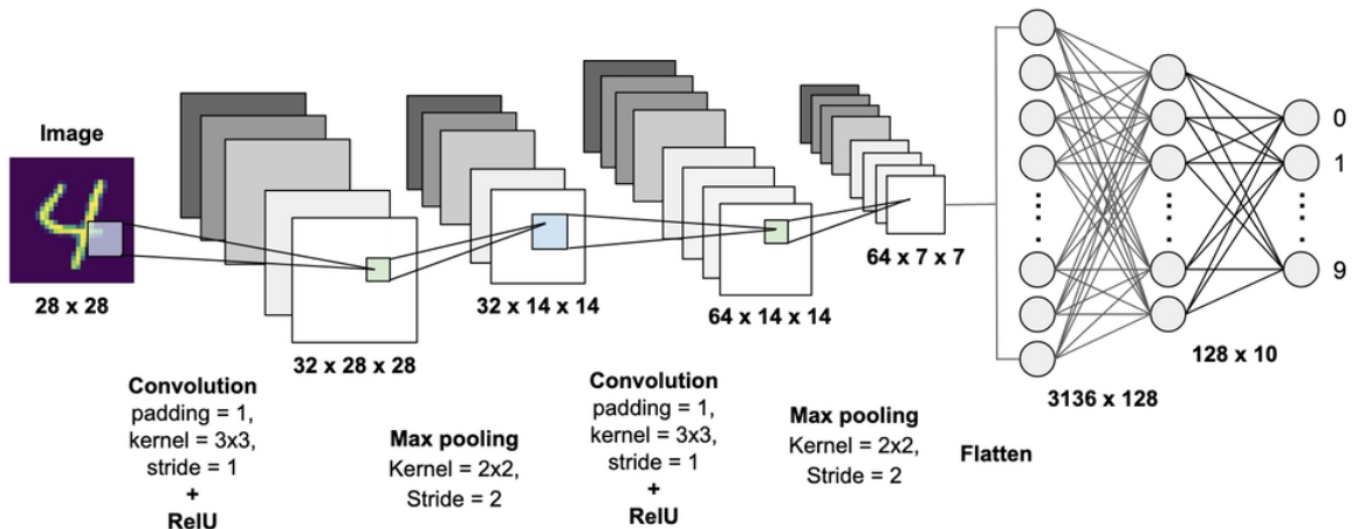
Advantages of CNN

- Learns features automatically
- Reduces number of parameters
- Best for image-related tasks
- Captures spatial information
- High accuracy compared to ANN

Applications of CNN

- Face recognition
- Object detection

- Medical imaging
- Autonomus vehicles
- Security camera systems
- Number plate recognition
- Emotion detection
- Satellite image analysis



```
# =====
# 1. Import Libraries
# =====
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt

# =====
# 2. Load MNIST Dataset
# =====
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()

# =====
# 3. Preprocessing
# =====
# Normalize pixel values (0-255 → 0-1)
x_train = x_train / 255.0
x_test = x_test / 255.0

# Add channel dimension (28,28 → 28,28,1)
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)
```

```
# =====  
# 4. Build CNN Model  
# =====  
model = models.Sequential([  
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),  
    layers.MaxPooling2D((2,2)),  
  
    layers.Conv2D(64, (3,3), activation='relu'),  
    layers.MaxPooling2D((2,2)),  
  
    layers.Flatten(),  
    layers.Dense(128, activation='relu'),  
    layers.Dense(10, activation='softmax')  
)  
  
# Print model summary  
model.summary()  
  
# =====  
# 5. Compile the Model  
# =====  
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])  
  
# =====  
# 6. Train the CNN  
# =====  
history = model.fit(x_train, y_train, epochs=5,  
                    validation_data=(x_test, y_test))  
  
# =====  
# 7. Evaluate the Model  
# =====  
test_loss, test_acc = model.evaluate(x_test, y_test)  
print("\nTest Accuracy:", test_acc)
```



```
# =====  
# 8. Plot Accuracy and Loss Curves  
# =====  
plt.figure(figsize=(12,4))  
  
# Accuracy Plot  
plt.subplot(1,2,1)  
plt.plot(history.history['accuracy'], label='Training Accuracy')  
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
plt.title('Accuracy')  
plt.legend()  
  
# Loss Plot  
plt.subplot(1,2,2)  
plt.plot(history.history['loss'], label='Training Loss')  
plt.plot(history.history['val_loss'], label='Validation Loss')  
plt.title('Loss')  
plt.legend()  
  
plt.show()  
  
# =====  
# 9. Predict a Single Image  
# =====  
import numpy as np  
  
index = 0 # change this to test another image  
img = x_test[index].reshape(1, 28, 28, 1)  
  
prediction = model.predict(img)  
print("Predicted Label:", np.argmax(prediction))  
  
plt.imshow(x_test[index].reshape(28,28), cmap='gray')  
plt.title("Actual Label: " + str(y_test[index]))  
plt.show()
```

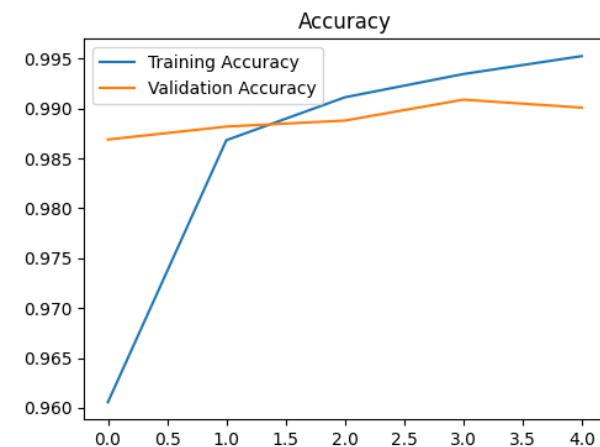
```
... Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 — 8s 8us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/5
1875/1875 — 36s 19ms/step - accuracy: 0.9103 - loss: 0.2969 - val_accuracy: 0.9869 - val_loss: 0.0447
Epoch 2/5
1875/1875 — 34s 18ms/step - accuracy: 0.9854 - loss: 0.0454 - val_accuracy: 0.9882 - val_loss: 0.0362
Epoch 3/5
1875/1875 — 41s 18ms/step - accuracy: 0.9911 - loss: 0.0282 - val_accuracy: 0.9888 - val_loss: 0.0350
Epoch 4/5
1875/1875 — 35s 19ms/step - accuracy: 0.9946 - loss: 0.0176 - val_accuracy: 0.9909 - val_loss: 0.0271
Epoch 5/5
1875/1875 — 41s 18ms/step - accuracy: 0.9963 - loss: 0.0116 - val_accuracy: 0.9901 - val_loss: 0.0354
313/313 — 3s 8ms/step - accuracy: 0.9864 - loss: 0.0467

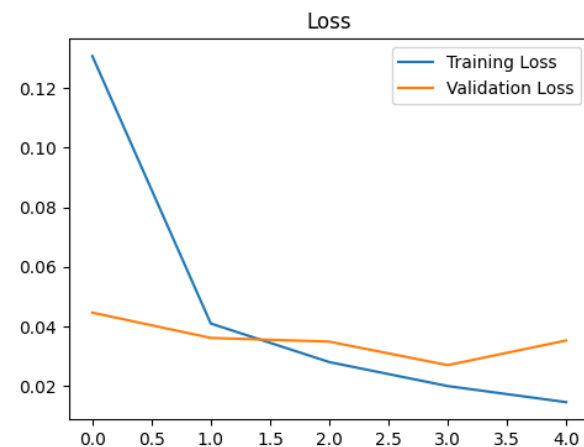
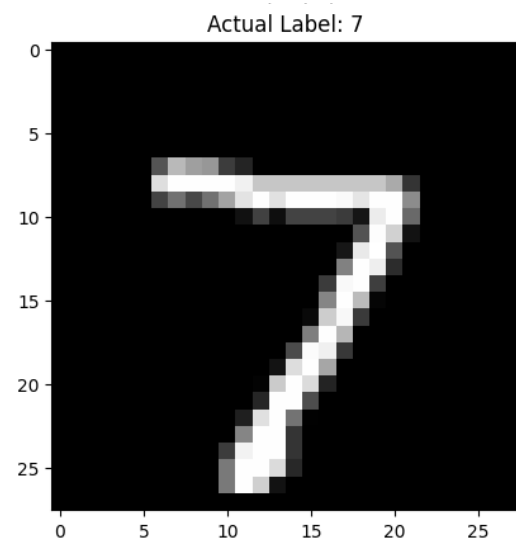
Test Accuracy: 0.9901000261306763

Test Accuracy: 0.9901000261306763



1/1 — 0s 69ms/step

Predicted Label: 7



Lab Task: Implementing a Convolutional Neural Network (CNN) on CIFAR-10 Dataset

1. Load and explore the CIFAR-10 dataset

- Display dataset shape
- Show at least **10 sample images**
- Visualize class names

2. Preprocess the data

- Normalize pixel values
- Convert labels if required (e.g. one-hot encoding)

3. Build a CNN model

Your CNN must include:

- At least **2 convolutional layers**
- **ReLU activation**
- **Max-pooling layers**
- **Dropout** (optional but recommended)
- **Fully connected (Dense) layers**
- **Softmax** output layer

4. Train the CNN

- Train for **at least 10 epochs**
- Show:
 - Training accuracy
 - Validation accuracy
 - Training loss
 - Validation loss

5. Evaluate the model

- Compute the **test accuracy**
- Show the **confusion matrix**
- Display **classification report** (precision, recall, F1-score)

6. Make predictions

- Pick 5 random CIFAR-10 test images
- Show each image with:
 - Actual label
 - Predicted label

7. Plot results

You must plot:

- Training vs validation accuracy
- Training vs validation loss

8. Provide analysis

Write a short paragraph explaining:

- How well the CNN performed
- Why accuracy is high/low
- What improvements can be made (e.g., more layers, dropout, data augmentation)